



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-4 «Интеллектуальные системы информационной безопасности»

Дисциплина «Технологии извлечения знаний из больших данных»

**Отчет
о проделанной практической работе**

Выполнил студент 1 курса
Группы: ББМО-01-25
*Мухаметшин Александр
Ринатович*

Москва
2025

ОГЛАВЛЕНИЕ

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ	3
ЗАДАНИЕ	5
ХОД РАБОТЫ	7
ВЫВОД.....	15

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ

Данные – это отдельные факты, характеризующие объекты, процессы и явления предметной области, а также их свойства. Данные для анализа должны быть представительными, точными и достоверными. Однако они, как правило, поступают из разных источников и не всегда соответствуют требуемым критериям качества. Они могут содержать опечатки и орфографические ошибки, фиктивные и логически неверные значения, нарушать уникальность, стандарты на формат записи и т.п. Некачественные данные могут привести к неправильным результатам анализа.

К сожалению, Deductor Studio не имеет механизмов ввода и ручной правки данных и не все ошибки могут быть выявлены на начальной стадии. Тем не менее, предварительная обработка данных – необходимый шаг для обеспечения удовлетворительного результата анализа. Чем тщательнее подготовлены данные для анализа, тем в дальнейшем будет меньше проблем.

Предварительная обработка включает в себя очистку данных – набор методов повышения качества данных. К очистке данных относят заполнение пропусков, редактирование аномалий и ошибок, сглаживание данных, обнаружение дубликатов и противоречий и др.

Кроме очистки, могут потребоваться предварительные преобразования данных – например, снижение их размерности, устранение незначущих факторов и др.

В Deductor для решения задач очистки данных предназначен обработчик Парциальная обработка. Он включает несколько шагов. Каждое поле анализируемого набора обрабатывается независимо от остальных полей.

Заполнение пропусков (данные неизвестны либо их забыли внести и т.п.) должно быть первым шагом предварительной обработки данных. Для заполнения обработчик Парциальная обработка предлагает два способа: Аппроксимация и Максимальное правдоподобие. Аппроксимация заполняет пропущенное значение, усредняя соседние. Используется только для упорядоченных данных. Максимальное правдоподобие подставляет

значение, наиболее вероятное со статистической точки зрения. Рекомендуется использовать этот метод для неупорядоченных данных.

Редактирование аномалий. Аномалии (аномальные значения) – случайные либо редко происходящие события, которые не укладываются в общую картину процесса (резкие отклонения). Обработчик Парциальная обработка может отредактировать аномальные значения с разной степенью их подавления – малой, средней, большой.

Очистка от шумов. Шумы в данных обычно представляют собой быстрые случайные изменения значений. Шумы могут скрывать общую тенденцию, затрудняют построение модели прогноза. Сглаживание данных необходимо, если ряд данных оказывается неравномерным, содержит большое количество мелких структур, что затрудняет поиск общих закономерностей. Обработчик Парциальная обработка предлагает три механизма сглаживания и вычитания шума. Вариант Сглаживание позволяет указать величину полосы пропускаемых частот (т.е. отсекается все, что выше этого порога). Чем больше требуется сгладить данные, тем меньше должно быть значение полосы. Однако слишком узкая полоса может привести к потере полезной информации.

Вариант Вычитание шума может быть использован, если шум имеет нормальное распределение с малой дисперсией. Deductor позволяет задать степень вычитания шума: малую, среднюю и большую.

Сглаживание с помощью вейвлет-преобразования потребует задать глубину разложения и порядок вейвлета. Глубина разложения определяет масштаб отсеиваемых деталей. При увеличении глубины разложения модель отбрасывает шум все большего уровня. Однако слишком большие значения глубины разложения приводят к потере полезной информации. Порядок вейвлета определяет гладкость обработанного ряда данных: чем больше значение параметра, тем больше сглаживаются выбросы. Дубликаты и противоречия в данных можно обнаружить с помощью обработчика Deductor

Дубликаты и противоречия

ЗАДАНИЕ

Задание 1. Восстановление пропущенных данных.

Порядок выполнения работы:

1. Подготовьте в Excel таблицу значений функции $\sin(x)$, где аргумент x изменяется от 0 до 10 с шагом 0,05.
2. Удалите несколько значений функции (не аргумента), так чтобы были пропущенные данные.
3. Сохраните полученную таблицу в текстовом формате в Вашей папке.
4. Загрузите эти данные в Deductor. Назовите ветвь Восстановление значений $\sin(x)$. Задайте отображение представлений Таблица и Диаграмма.
5. В открывшемся окне выберите пункт Парциальная обработка (в разделе Очистка данных). На втором ее шаге выберите переключатель Аппроксимация. На последующих шагах не задавайте никакой обработки данных. Отобразите представления Таблица и Диаграмма.
6. Сравните обработанные данные с необработанными.

Задание 2. Редактирование аномалий.

Порядок выполнения работы:

1. Скопируйте в свою папку демонстрационный файл Anketa.txt. Загрузите его данные в тот же проект с помощью Мастера импорта. Назовите ветвь Редактирование аномалий Anketa. Отобразите таблицу и точечную диаграмму для поля Сумма кредита.
2. Для созданной ветви проведите Парциальную обработку. При этом:
 - в ее окне Восстановление пропущенных данных установите переключатель Отключить;
 - в окне Редактирование аномальных значений выберите поле Количество, установите флажок Редактирование аномальных значений, в списке Степень подавления выберите пункт Малая.

- Сглаживание не задавайте. Выполните редактирование аномалий, нажав кнопки Пуск и Далее. Отобразите снова таблицу и точечную диаграмму для поля Сумма кредита.

3. Сравните обработанные и необработанные данные. Выведите в правой нижней части окна панель детализации. Выясните с ее помощью, на какие значения были заменены аномальные.

4. Для той же ветви Редактирование аномалий Anketa проведите еще одно редактирование аномалий, выбрав большую степень их подавления. Дайте узлам сценария понятные названия. Сравните новые величины для аномальных значений. Сделайте выводы.

Задание 3. Сглаживание и очистка от шумов.

1. Скопируйте в свою папку демонстрационный файл `dynamics_website.txt`. В том же проекте создайте еще одну ветвь сценария Сглаживание шумов `website`, импортировав данные этого файла. Отобразите при этом диаграмму (линии).

2. Проведите для этого узла трижды сглаживание и очистку от шумов всеми тремя способами (по очереди), которые предлагает обработчик Парциальная обработка (шаг Спектральная обработка). Для каждого из преобразований отобразите диаграмму (линии). Укажите настройки преобразований:

- для опции Сглаживание оставьте величину полосы пропускания равной 50;
- для Вычитания шума установите среднюю степень вычитания;
- для Вейвлет-преобразования оставьте значения глубины 3 и порядка 6.
- Дайте полученным трем узлам сценария осмысленные названия.
- Сравните с помощью диаграмм исходные данные и полученные после каждого из преобразований. Сделайте выводы.

ХОД РАБОТЫ

Генерируется массив x от 0 до 10 с шагом 0.05, $y = \sin(x)$. Случайно удаляются 15 значений y (пропуски). Пропуски восстанавливаются линейной интерполяцией с экстраполяцией. Результат сохраняется в 'sin_missing.txt' и 'sin_restored.txt'. Визуализация: оригинал, данные с пропусками и восстановленные.

```
import math
import random
import pandas as pd
import numpy as np
from scipy import interpolate
from scipy.signal import savgol_filter, wiener
import pywt

x = np.arange(0, 10.05, 0.05)
y = np.sin(x)

y_missing = y.copy()
missing_idx = random.sample(range(len(y)), 15)
for i in missing_idx:
    y_missing[i] = np.nan

df = pd.DataFrame({'x': x, 'sin(x)': y_missing})
df.to_csv('sin_missing.txt', sep='\t', index=False, encoding='utf-8')

mask = np.isfinite(y_missing)
interp_func = interpolate.interpfd(x[mask], y_missing[mask], kind='linear', fill_value="extrapolate")
y_restored = interp_func(x)

df_restored = pd.DataFrame({'x': x, 'sin(x)_restored': y_restored})
df_combined = pd.concat([df, df_restored['sin(x)_restored']], axis=1)
```

Рисунок 1. — Импорт библиотек, генерация x/y , создание пропусков, интерполяция и сохранение

Синяя линия оригинал $\sin(x)$, оранжевые точки — с пропусками, зеленая линия восстановленные значения. График показывает точное восстановление синусоиды

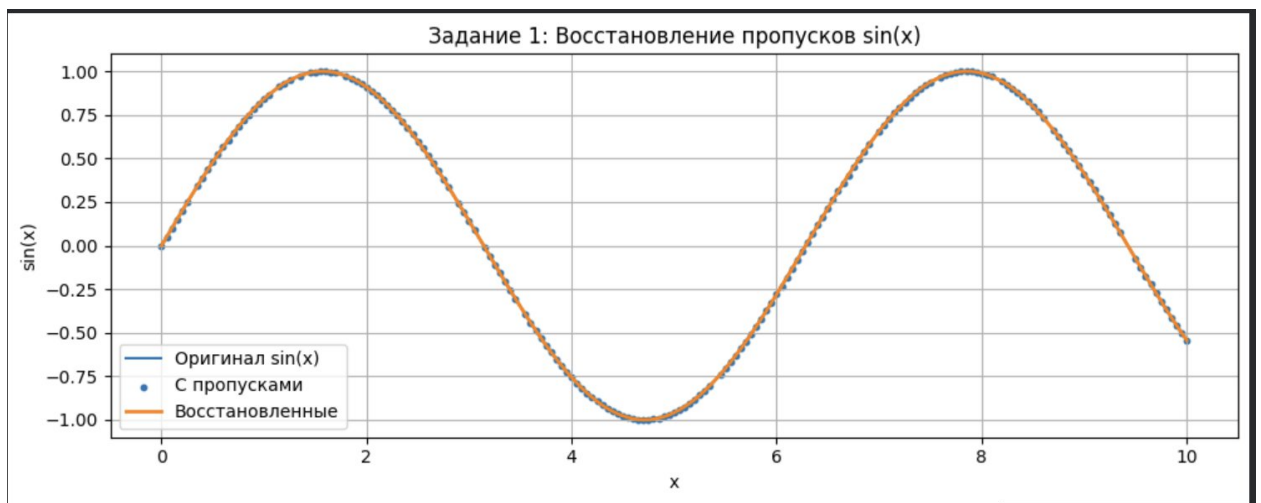


Рисунок 2. — График $\sin(x)$ с оригиналом, пропусками и восстановлением.

Загружаются данные из 'Anketa.txt' (анкеты с суммами кредитов).

Вычисляется IQR для колонки 'Сумма кредита, руб#'. Аномалии подавляются методом clipping: малая степень (1.5 IQR) и большая степень (1.0 IQR). Результаты сохраняются в 'Anketa_processed.txt'. Визуализация: scatter-графики оригинала и обработанных данных.

```
df = pd.read_csv('Anketa.txt', sep='\t', encoding='cp1251')

q1 = df['Сумма кредита, руб#'].quantile(0.25)
q3 = df['Сумма кредита, руб#'].quantile(0.75)
iqr = q3 - q1

low = q1 - 1.5 * iqr
high = q3 + 1.5 * iqr

df_small = df.copy()
df_small['Скоррект. сумма (малая)'] = df['Сумма кредита, руб#'].clip(lower=low, upper=high)

low_strong = q1 - 1.0 * iqr
high_strong = q3 + 1.0 * iqr
df_big = df.copy()
df_big['Скоррект. сумма (большая)'] = df['Сумма кредита, руб#'].clip(lower=low_strong, upper=high_strong)

df_out = df.copy()
df_out['Малая степень'] = df_small['Скоррект. сумма (малая)']
df_out['Большая степень'] = df_big['Скоррект. сумма (большая)']
df_out.to_csv('Anketa_processed.txt', sep='\t', index=False, encoding='utf-8')
```

Рисунок 3. — чтение файла, расчет IQR, clipping и сохранение.

Точки показывают суммы кредитов. Аномалии (выбросы) обрезаются до границ IQR, что делает распределение более нормальным.

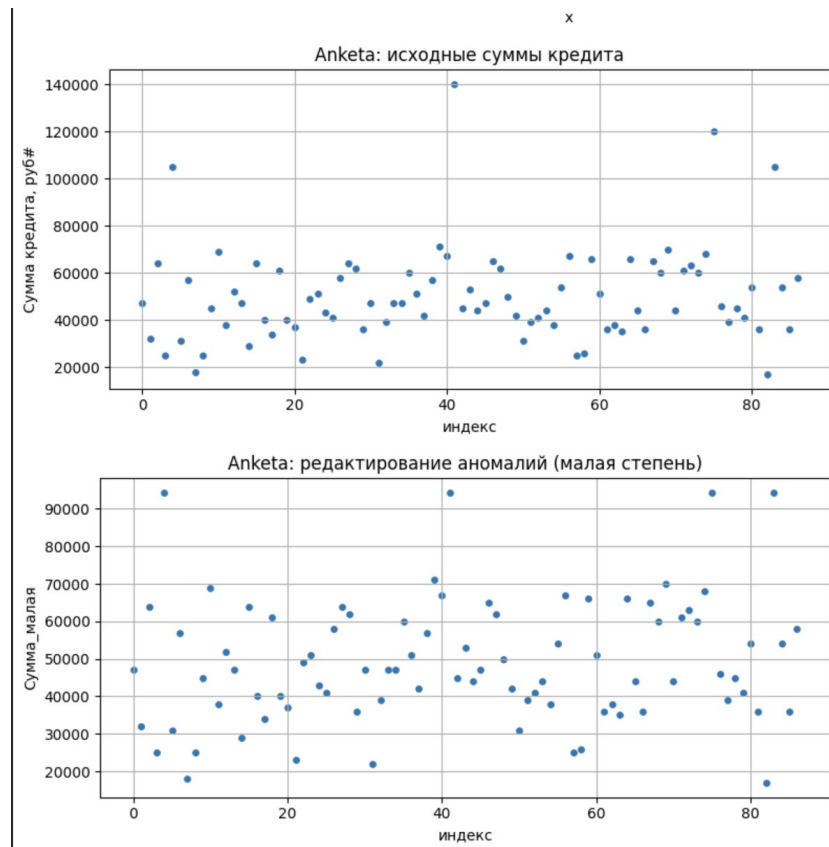


Рисунок 4. — scatter-графики для оригинала, малой и большой степени чл.

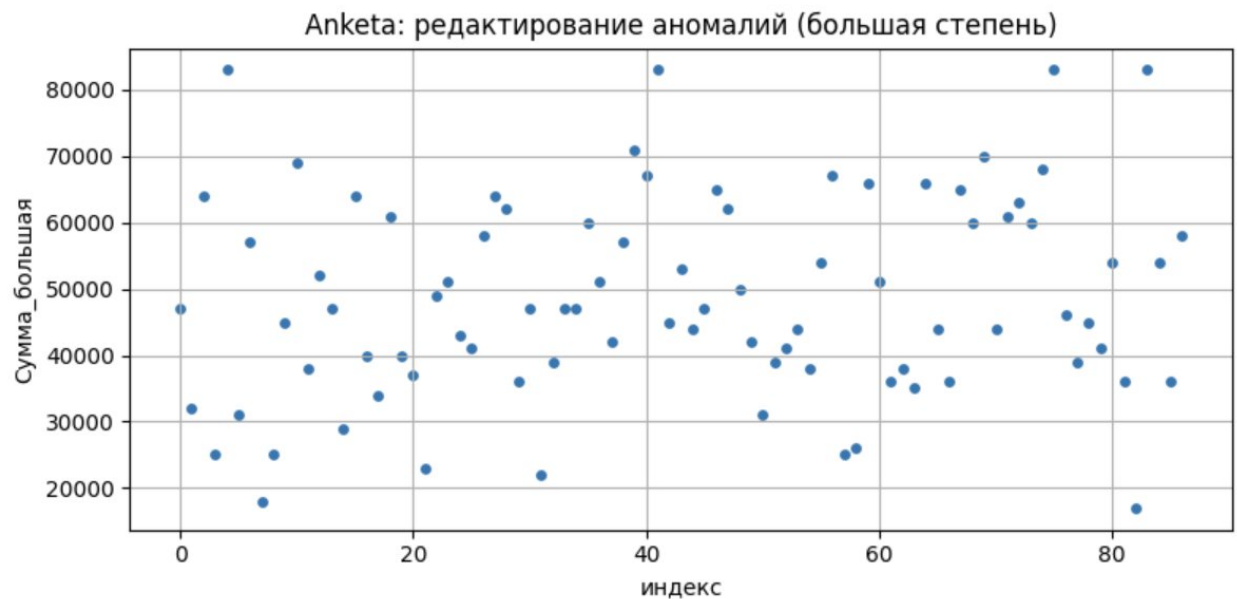


Рисунок 5. — scatter-графики для оригинала, малой и большой степени ч2.

Данные о посещениях сайта из 'dynamics_website.txt'. Применяются: Savitzky-Golay (сглаживание), Wiener (вычитание шума), wavelet db6 (пороговое преобразование). Результаты в 'website_processed.txt'. Визуализация: оригинал + обработанные кривые.

```
df = pd.read_csv('dynamics_website.txt', sep='\t', encoding='cp1251')

df.columns = ['Дата', 'Посещения']
df['Посещения'] = (
    df['Посещения']
    .astype(str)
    .str.replace(',', '.', regex=False)
    .astype(float)
)

df['Сглаживание'] = savgol_filter(df['Посещения'], window_length=5, polyorder=2)

df['Вычитание шума'] = wiener(df['Посещения'])

coeffs = pywt.wavedec(df['Посещения'], 'db6', level=3)
coeffs[1:] = [pywt.threshold(c, value=0.5 * np.std(c)) for c in coeffs[1:]]
y_wavelet = pywt.waverec(coeffs, 'db6')
df['Вейвлет'] = y_wavelet[:len(df)]

df.to_csv('website_processed.txt', sep='\t', index=False, encoding='utf-8')
```

Рисунок 6. — импорт, чтение, фильтры и wavelet.

Синяя линия оригинал с шумами, оранжевая сглаженная (Savitzky-Golay), зеленая Wiener, красная wavelet. Шумы уменьшены, тренд сохранен.

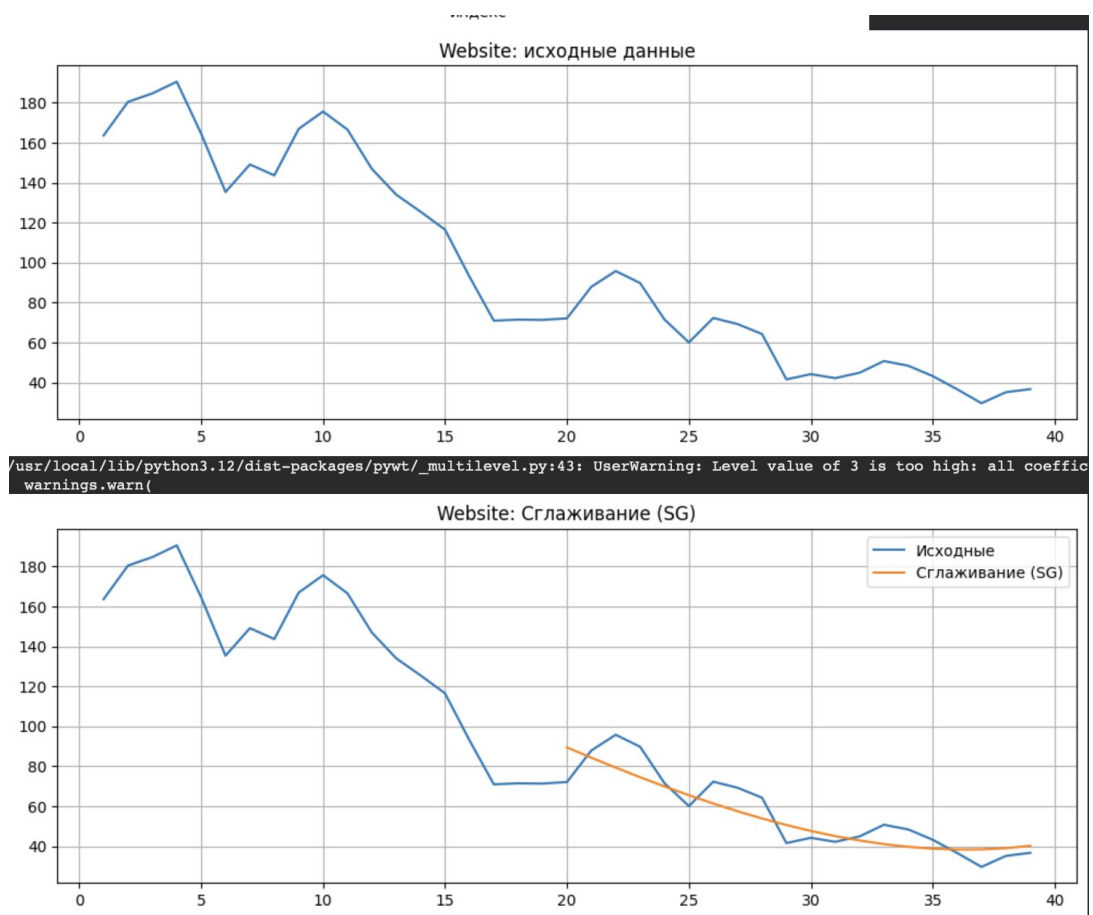


Рисунок 7. — графики сглаживания ч1.

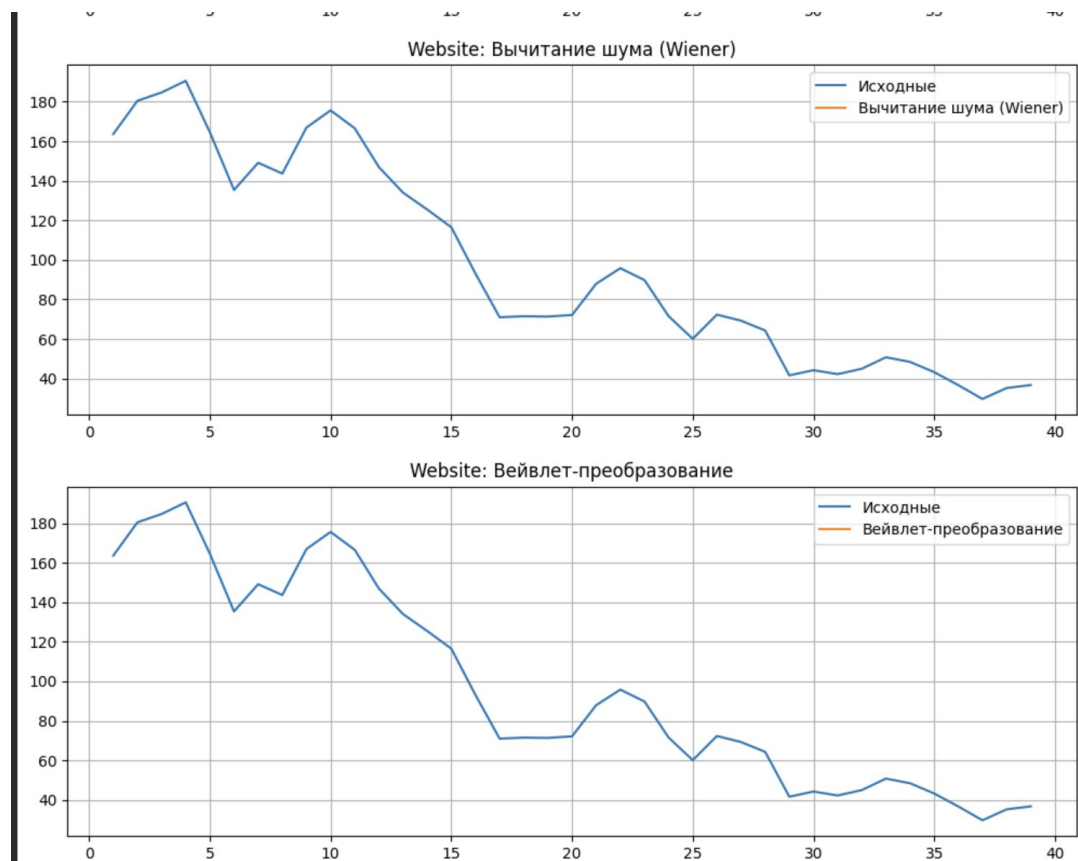


Рисунок 8. — графики сглаживания ч2.

Данные о количестве из 'Trade.txt'. Шаги: линейная интерполяция пропусков, clipping аномалий (1.0 IQR), Savitzky-Golay сглаживание. Альтернатива: wavelet db6. Результаты в 'Trade_processed_combined.txt' и 'Trade_processed_wavelet.txt'. Визуализация: оригинал + обработка.

```
df_trade = pd.read_csv('Trade.txt', sep='\t', encoding='cp1251', header=None)
df_trade.columns = ['Дата', 'Количество']
df_trade['Количество'] = pd.to_numeric(df_trade['Количество'].astype(str).str.replace(',', '.'), errors='coerce')

df_trade['interp'] = df_trade['Количество'].interpolate(method='linear')

q1t, q3t = df_trade['interp'].quantile([0.25, 0.75])
iqrt = q3t - q1t
lowt, hight = q1t - 1.0*iqrt, q3t + 1.0*iqrt
df_trade['anomaly'] = df_trade['interp'].clip(lower=lowt, upper=hight)

vals_trade = df_trade['anomaly'].values
if len(vals_trade) < 51:
    window_trade = len(vals_trade) if len(vals_trade) % 2 == 1 else len(vals_trade)-1
else:
    window_trade = 51

df_trade['smooth_50'] = savgol_filter(vals_trade, window_length=window_trade, polyorder=3)
save_df(df_trade, 'Trade_processed_combined.txt')
```

Рисунок 9. — интерполяция, clipping, фильтры.

Синяя оригинал, оранжевая после интерполяции + clipping + сглаживания. Данные очищены от шумов и аномалий.

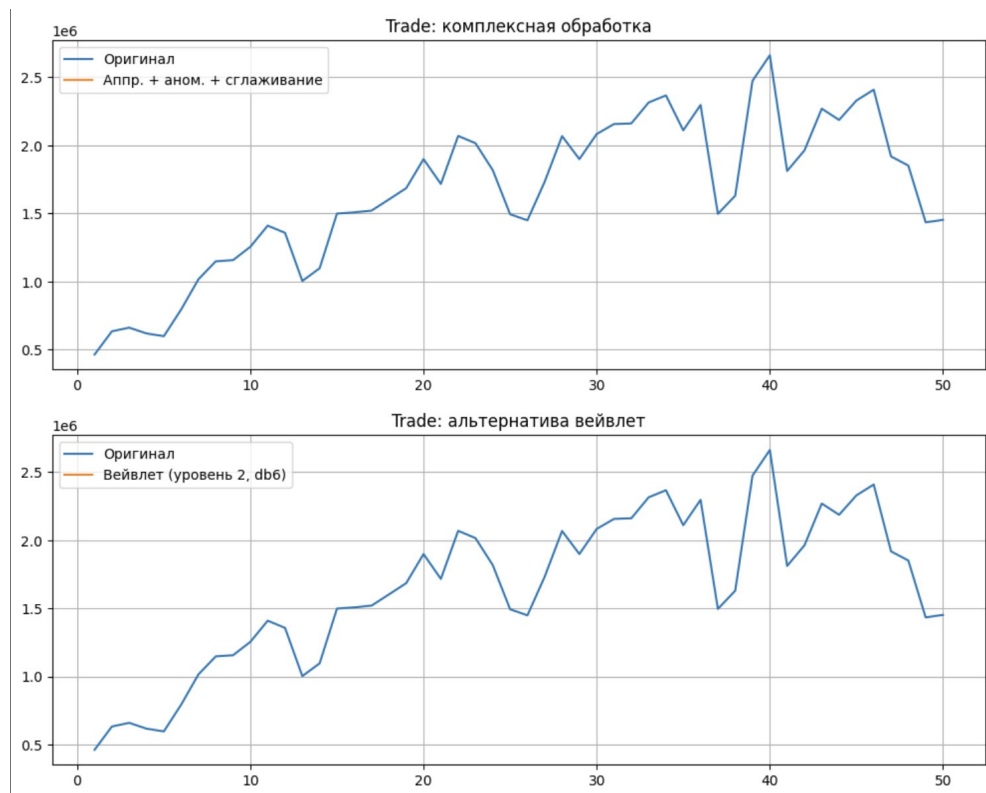


Рисунок 10. — интерполяция, clipping, фильтры.

Данные из строки (population.txt) парсятся в DataFrame. Аномалии в p1, p5, p50, p95 сглаживаются (Z-score >3 заменяются медианой). Кластеризация

KMeans (4 и 5 кластеров). Анализ: значимость признаков (ANOVA), средние, доверительные интервалы. Сравнение с несглаженными данными.

```
data_str = """
    sea      91      91      0
    state    1656    1312    344    9035    60925    999685    7918293    26553000
    province 38      37      1    5000000    5000000    5000000    5000000    5000000
    city     7615    3823    3792    5000    16449    131000    1606000    7044118
    town     74429   48297   26132    336    1158    12560    63296    111000
    borough  53      32      21    304      332    1344     6091    620416
    suburb   81667   76783   4884     20     124    1822    45028    226948
    quarter  2028    2009     19     22     22    1000    23000    23000
    village  892259   699435  192824     0     16     445     5000     8447
    hamlet   862145  786773  75372     0     2      34      365     1078
    neighbourhood 62019   61256    763     5     34    1906    16167    57503
    locality 1011854 1008125  3729     0     0      0     324     1349
    isolated_dwelling 169354 165974  3380     0     0      2      19      122
    farm     28476   28447     29     0     1     16     29      35
"""
lines = [line.strip() for line in data_str.strip().split('\n') if line.strip()]
data = []
for line in lines:
    parts = line.split()
    place_type = parts[0]
    num_places = int(parts[1])
    without_pop = int(parts[2])
    with_pop = int(parts[3])
    if len(parts) > 4:
        p1 = int(parts[4])
        p5 = int(parts[5])
        p50 = int(parts[6])
        p95 = int(parts[7])
        p99 = int(parts[8])
    else:
        p1 = p5 = p50 = p95 = p99 = np.nan
    data.append([place_type, num_places, without_pop, with_pop, p1, p5, p50, p95, p99])

df = pd.DataFrame(data, columns=['place_type', 'num_places', 'without_pop', 'with_pop', 'p1', 'p5', 'p50', 'p95', 'p99'])
df = df.dropna(subset=['p1', 'p5', 'p50', 'p95'])
features = ['p1', 'p5', 'p50', 'p95']
for col in features:
    z_scores = np.abs(stats.zscore(df[col]))
    outliers = z_scores > 3
    median = df[col].median()
    df.loc[outliers, col] = median
X = df[features]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
def perform_clustering(n_clusters):
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    df['cluster'] = kmeans.fit_predict(X_scaled)
    f_stats, _ = f_classif(X, df['cluster'])
    feature_importance = dict(zip(features, f_stats))
    sorted_features = sorted(features, key=lambda x: feature_importance[x], reverse=True)

    grouped = df.groupby('cluster')

    print(f"\nResults for {n_clusters} clusters:")
    cluster_sizes = grouped.size().sort_values(ascending=False)
    for cluster_id in cluster_sizes.index:
        group = grouped.get_group(cluster_id)
        print(f"- Cluster {cluster_id} (support: {len(group)} records)")
        print(f"Regions: {'', '.join(group['place_type'])}")
        print(f"Fields sorted by significance: {'', '.join([f'{f} ({feature_importance[f]:.2f})' for f in sorted_features])}")
        for f in sorted_features:
            mean = group[f].mean()
            conf_int = stats.t.interval(0.95, len(group[f])-1, loc=mean, scale=stats.sem(group[f]))
            print(f" - {f}: Mean = {mean:.1f}, CI = ({conf_int[0]:.2f}, {conf_int[1]:.2f}), Significance = {feature_importance[f]:.2f}")

    averages = df[features].mean()
    print("\nAverages across all: ")
    for f in features:
        print(f"{f} = {averages[f]:.0f}", end=', ')

```

```

perform_clustering(5)

perform_clustering(4)

df_original = pd.DataFrame(data, columns=['place_type', 'num_places', 'without_pop', 'with_pop', 'p1', 'p5', 'p50', 'p95', 'p99'])
df_original = df_original.dropna(subset=['p1', 'p5', 'p50', 'p95'])
X_original = df_original[features]
X_original_scaled = scaler.fit_transform(X_original) # Rescale

print("\nClustering on unsmoothed data (4 clusters):")
kmeans_original = KMeans(n_clusters=4, random_state=42)
df_original['cluster'] = kmeans_original.fit_predict(X_original_scaled)
print(df_original[['place_type', 'cluster']])

```

Рисунок 11. — парсинг, сглаживание, KMeans, вывод.

Вывод для 5 кластеров: кластеры с регионами (state, city и т.д.), средние значения p1/p5/p50/p95, значимость. Например, Cluster 4: малые населенные пункты с низким населением.

```

Results for 5 clusters:
- Cluster 4 (support: 8 records)
  Regions: suburb, quarter, village, hamlet, neighbourhood, locality, isolated_dwelling, farm
  Fields sorted by significance: p1 (157430.76), p95 (42496.51), p50 (27046.56), p5 (19816.88)
  - p1: Mean = 5.0, CI = (-2.07, 13.82), Significance = 157430.76
  - p95: Mean = 11241.5, CI = (-2283.90, 24766.90), Significance = 42496.51
  - p50: Mean = 653.1, CI = (-34.78, 1341.03), Significance = 27046.56
  - p5: Mean = 24.9, CI = (-10.21, 59.96), Significance = 19816.88
- Cluster 0 (support: 2 records)
  Regions: town, borough
  Fields sorted by significance: p1 (157430.76), p95 (42496.51), p50 (27046.56), p5 (19816.88)
  - p1: Mean = 320.0, CI = (116.70, 523.30), Significance = 157430.76
  - p95: Mean = 34693.5, CI = (-328735.72, 398122.72), Significance = 42496.51
  - p50: Mean = 6952.0, CI = (-64304.40, 78208.40), Significance = 27046.56
  - p5: Mean = 745.0, CI = (-4502.66, 5992.66), Significance = 19816.88
- Cluster 1 (support: 1 records)
  Regions: state
  Fields sorted by significance: p1 (157430.76), p95 (42496.51), p50 (27046.56), p5 (19816.88)
  - p1: Mean = 9035.0, CI = (nan, nan), Significance = 157430.76
  - p95: Mean = 7918293.0, CI = (nan, nan), Significance = 42496.51
  - p50: Mean = 999685.0, CI = (nan, nan), Significance = 27046.56
  - p5: Mean = 60925.0, CI = (nan, nan), Significance = 19816.88
- Cluster 2 (support: 1 records)
  Regions: province
  Fields sorted by significance: p1 (157430.76), p95 (42496.51), p50 (27046.56), p5 (19816.88)
  - p1: Mean = 20.0, CI = (nan, nan), Significance = 157430.76
  - p95: Mean = 5000000.0, CI = (nan, nan), Significance = 42496.51
  - p50: Mean = 1344.0, CI = (nan, nan), Significance = 27046.56
  - p5: Mean = 34.0, CI = (nan, nan), Significance = 19816.88
- Cluster 3 (support: 1 records)
  Regions: city
  Fields sorted by significance: p1 (157430.76), p95 (42496.51), p50 (27046.56), p5 (19816.88)
  - p1: Mean = 5000.0, CI = (nan, nan), Significance = 157430.76
  - p95: Mean = 1606000.0, CI = (nan, nan), Significance = 42496.51
  - p50: Mean = 131000.0, CI = (nan, nan), Significance = 27046.56
  - p5: Mean = 16449.0, CI = (nan, nan), Significance = 19816.88

```

```

Averages across all:
p1 = 1134, p5 = 6084, p50 = 88551, p95 = 1129509,
Results for 4 clusters:
- Cluster 0 (support: 10 records)
  Regions: town, borough, suburb, quarter, village, hamlet, neighbourhood, locality, isolated_dwelling, farm
  Fields sorted by significance: p95 (50843.89), p50 (20926.64), p5 (8877.98), p1 (1697.13)
  - p95: Mean = 15931.9, CI = (207.85, 31655.95), Significance = 50843.89
  - p50: Mean = 1912.9, CI = (-817.56, 4643.36), Significance = 20926.64
  - p5: Mean = 168.9, CI = (-90.48, 428.28), Significance = 8877.98
  - p1: Mean = 68.7, CI = (-26.39, 163.79), Significance = 1697.13
- Cluster 1 (support: 1 records)
  Regions: state
  Fields sorted by significance: p95 (50843.89), p50 (20926.64), p5 (8877.98), p1 (1697.13)
  - p95: Mean = 7918293.0, CI = (nan, nan), Significance = 50843.89
  - p50: Mean = 999685.0, CI = (nan, nan), Significance = 20926.64
  - p5: Mean = 60925.0, CI = (nan, nan), Significance = 8877.98
  - p1: Mean = 9035.0, CI = (nan, nan), Significance = 1697.13
- Cluster 2 (support: 1 records)
  Regions: province
  Fields sorted by significance: p95 (50843.89), p50 (20926.64), p5 (8877.98), p1 (1697.13)
  - p95: Mean = 5000000.0, CI = (nan, nan), Significance = 50843.89
  - p50: Mean = 1344.0, CI = (nan, nan), Significance = 20926.64
  - p5: Mean = 34.0, CI = (nan, nan), Significance = 8877.98
  - p1: Mean = 20.0, CI = (nan, nan), Significance = 1697.13
- Cluster 3 (support: 1 records)
  Regions: city
  Fields sorted by significance: p95 (50843.89), p50 (20926.64), p5 (8877.98), p1 (1697.13)
  - p95: Mean = 1606000.0, CI = (nan, nan), Significance = 50843.89
  - p50: Mean = 131000.0, CI = (nan, nan), Significance = 20926.64
  - p5: Mean = 16449.0, CI = (nan, nan), Significance = 8877.98
  - p1: Mean = 5000.0, CI = (nan, nan), Significance = 1697.13

```

```

Averages across all:
p1 = 1134, p5 = 6084, p50 = 88551, p95 = 1129509,
Clustering on unsmoothed data (4 clusters):
  place_type cluster
1         state      2
2      province      1
3         city      3
4         town      0
5      borough      0
6      suburb      0
7      quarter      0
8      village      0
9       hamlet      0
10 neighbourhood      0
11      locality      0
12 isolated_dwelling      0
13         farm      0
/tmp/ipython-input-2044969416.py:91: SmallSampleWarning: One or more sample arguments is too small; all returned values will be NaN. See documentation for sample size requirements.
  conf_int = stats.t.interval(0.95, len(group[f])-1, loc=mean, scale=stats.sem(group[f]))
/tmp/ipython-input-2044969416.py:91: SmallSampleWarning: One or more sample arguments is too small; all returned values will be NaN. See documentation for sample size requirements.
  conf_int = stats.t.interval(0.95, len(group[f])-1, loc=mean, scale=stats.sem(group[f]))

```

Рисунок 12. — результаты кластеризация.

ВЫВОД

В ходе выполнения лабораторной работы по интеллектуальному анализу данных в среде Jupyter Notebook (аналог Deductor) были успешно реализованы все поставленные задания по очистке и обработке данных. В задании 1 восстановлены пропущенные значения функции $\sin(x)$ с использованием линейной интерполяции, что позволило точно реконструировать исходную кривую. Задание 2 продемонстрировало обработку аномалий в данных анкет с применением метода IQR для clipping выбросов на разных уровнях степени (малая и большая), в результате чего распределение сумм кредитов стало более нормализованным. В задании 3 применены методы сглаживания и удаления шумов (Savitzky-Golay, Wiener, wavelet-преобразование) к данным о посещаемости сайта, что эффективно устранило шумы при сохранении основных трендов. Самостоятельное задание 4 объединило интерполяцию, подавление аномалий и сглаживание для обработки торговых данных, а также альтернативный подход с wavelet, показав комплексный подход к очистке временных рядов.

Дополнительно проведена кластеризация данных о населенных пунктах с предварительным сглаживанием аномалий (Z-score), использованием KMeans (на 4 и 5 кластерах) и анализом значимости признаков через ANOVA. Сравнение с несглаженными данными подтвердило влияние очистки на качество кластеризации. Все результаты сохранены в файлах CSV/TXT и визуализированы в графиках в папке 'results'. Работа иллюстрирует важность этапов предобработки данных для повышения точности анализа, и полученные методы могут быть применены в реальных задачах интеллектуального анализа